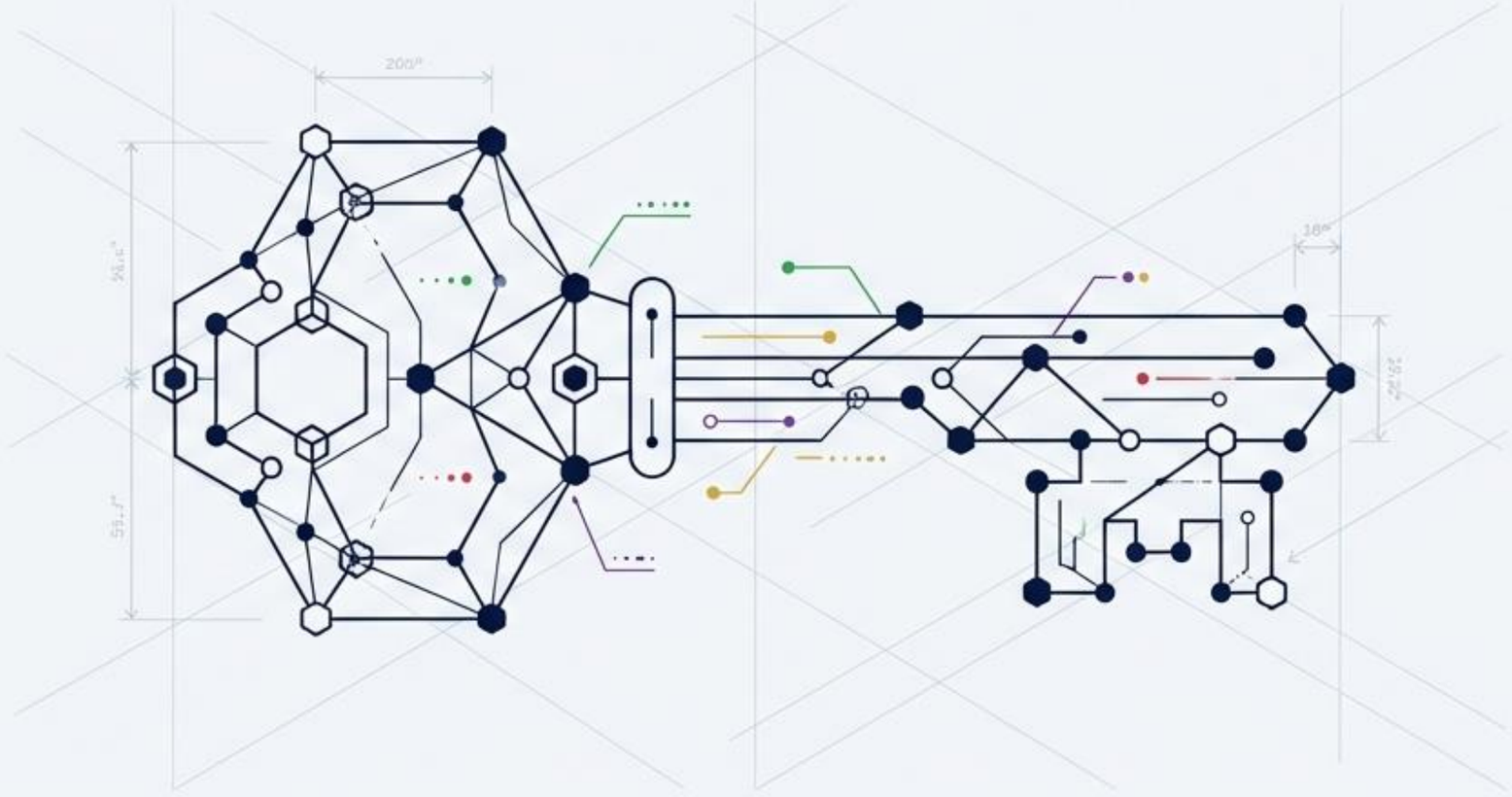
# Architecture de la Confiance : PKI, TLS et Cryptographie

De la norme ISO 27001 aux mathématiques RSA

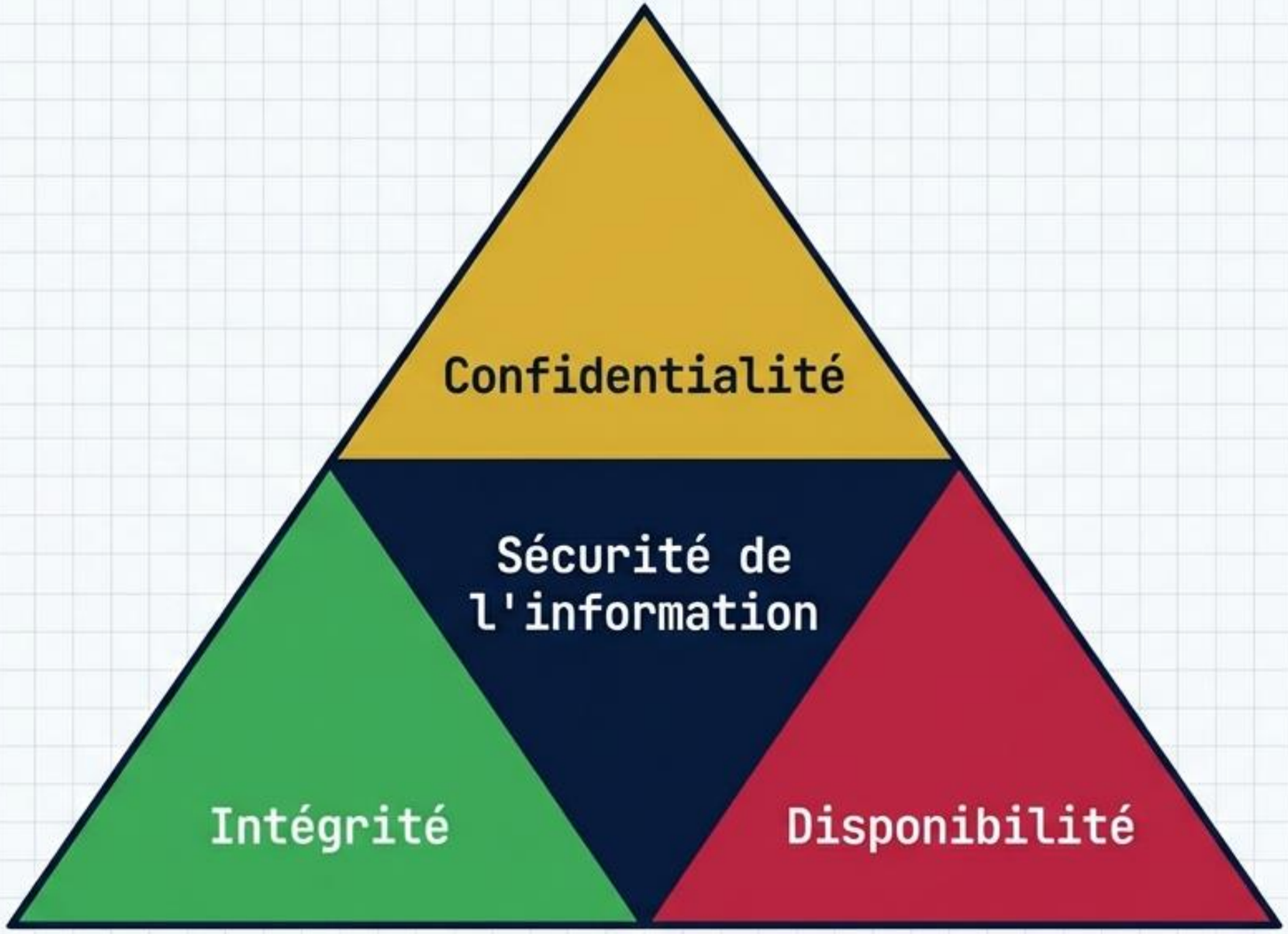


# LE CADRE DE GOUVERNANCE : ISO/IEC 27001

**L'ISO/IEC 27001 EST UNE NORME INTERNATIONALE FOURNISSANT UN CADRE POUR PROTÉGER LES INFORMATIONS SENSIBLES.**

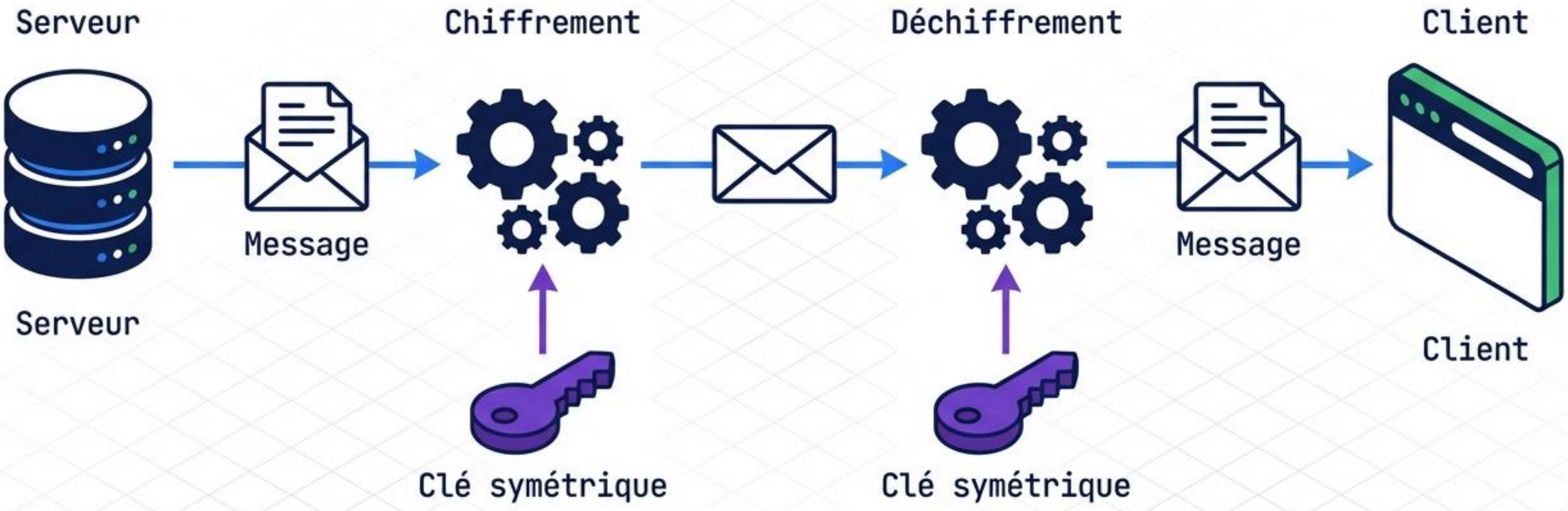
- **OBJECTIF :**
  - DÉFENDRE L'ORGANISATION CONTRE LES CYBERMENACES ET LES VIOLATIONS DE DONNÉES VIA DES CONTRÔLES DE SÉCURITÉ ADAPTÉS.

# LA TRIADE DE LA SÉCURITÉ




Ces trois piliers dictent toutes les implémentations techniques qui suivront.

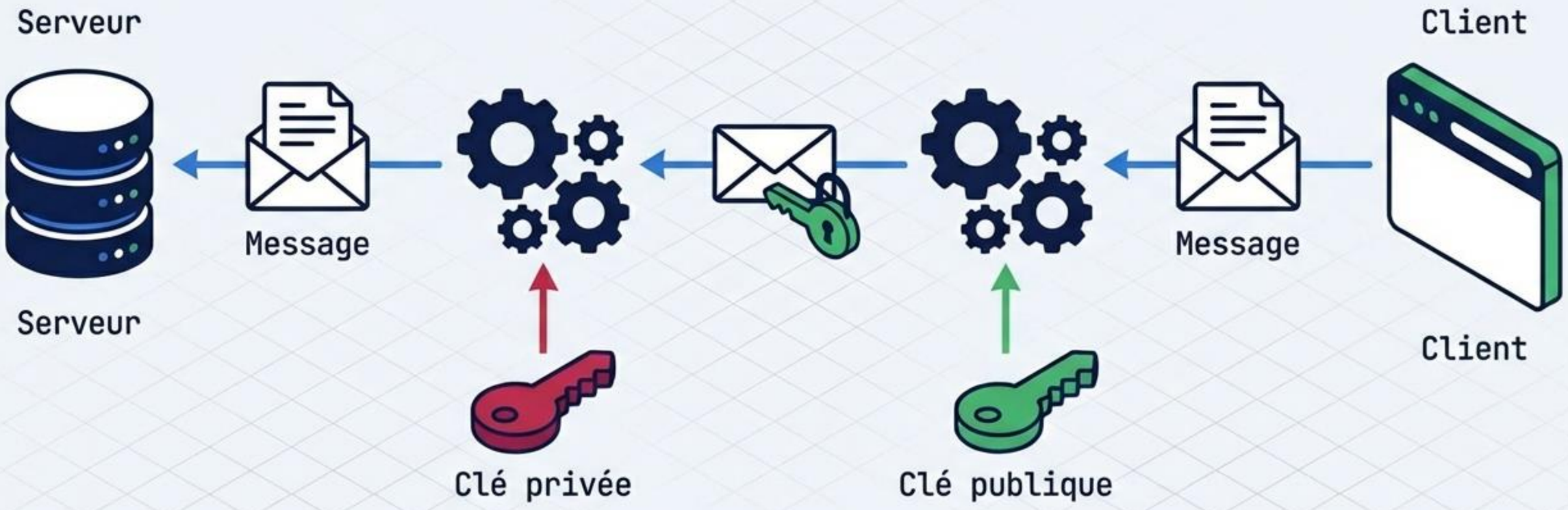
# LE CHIFFREMENT SYMÉTRIQUE




Exemple : Algorithme AES

⚠ Rapide, mais comment échanger la clé en toute sécurité ?

# LE CHIFFREMENT ASYMÉTRIQUE



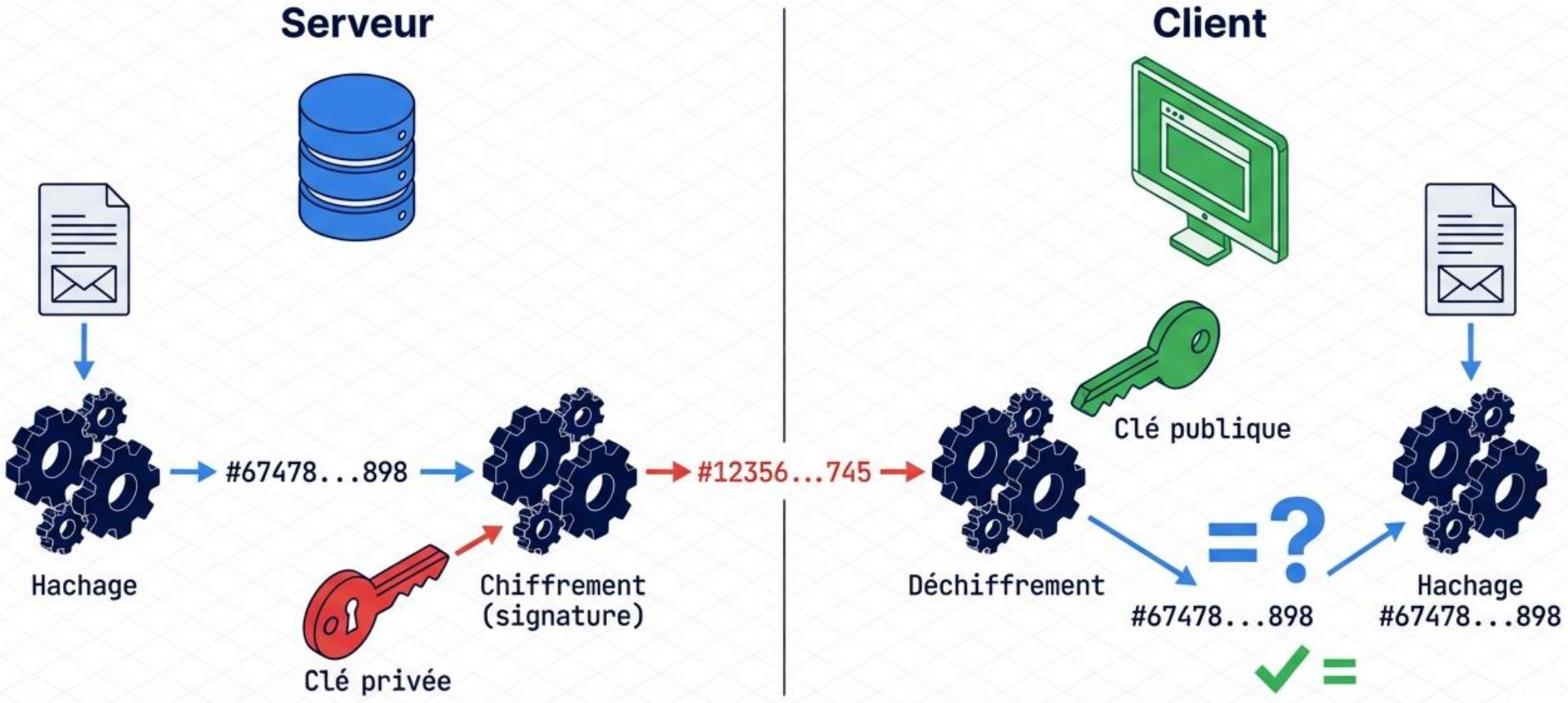

Exemple : Algorithme RSA

# LE HACHAGE (HASHING)

Objectif : vérifier l'intégrité

# La Signature Numérique

Une signature est un hachage chiffré avec la clé privée de l'émetteur.

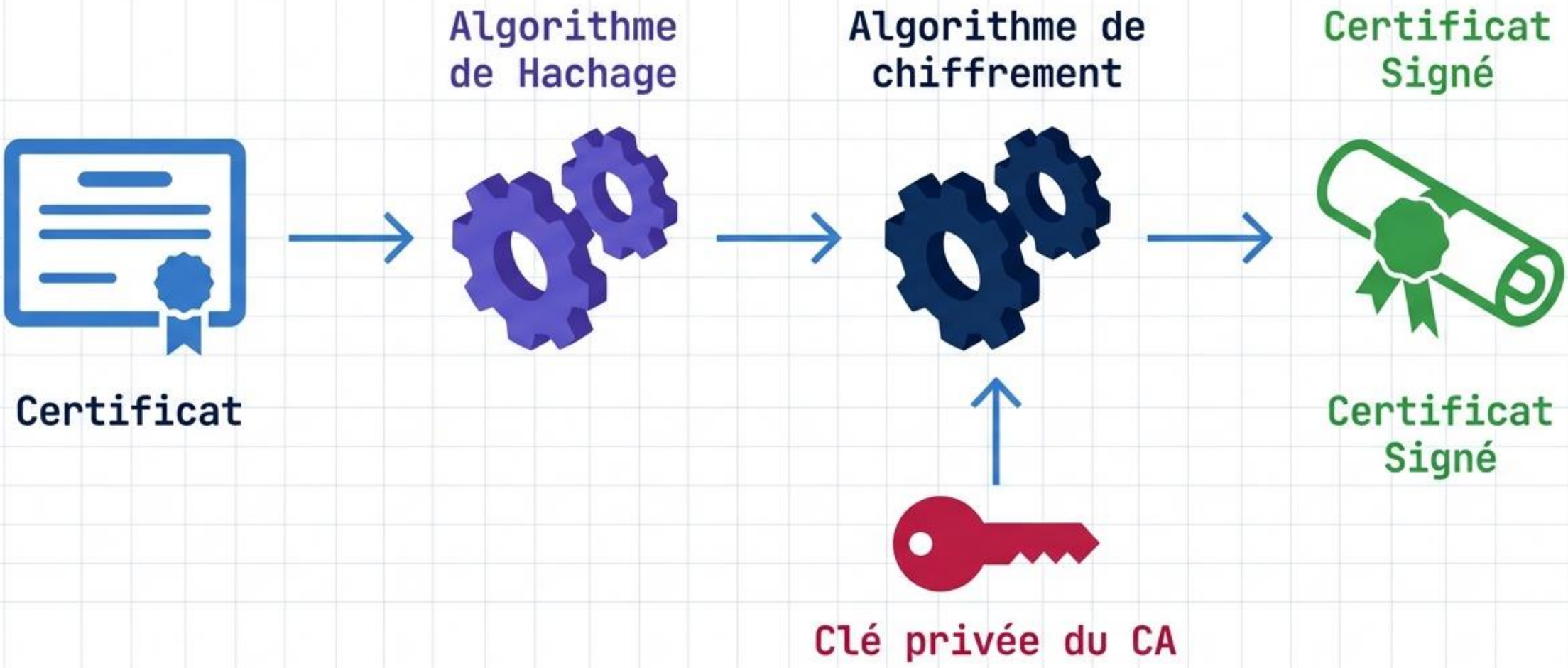


# CA : Certificate Authority

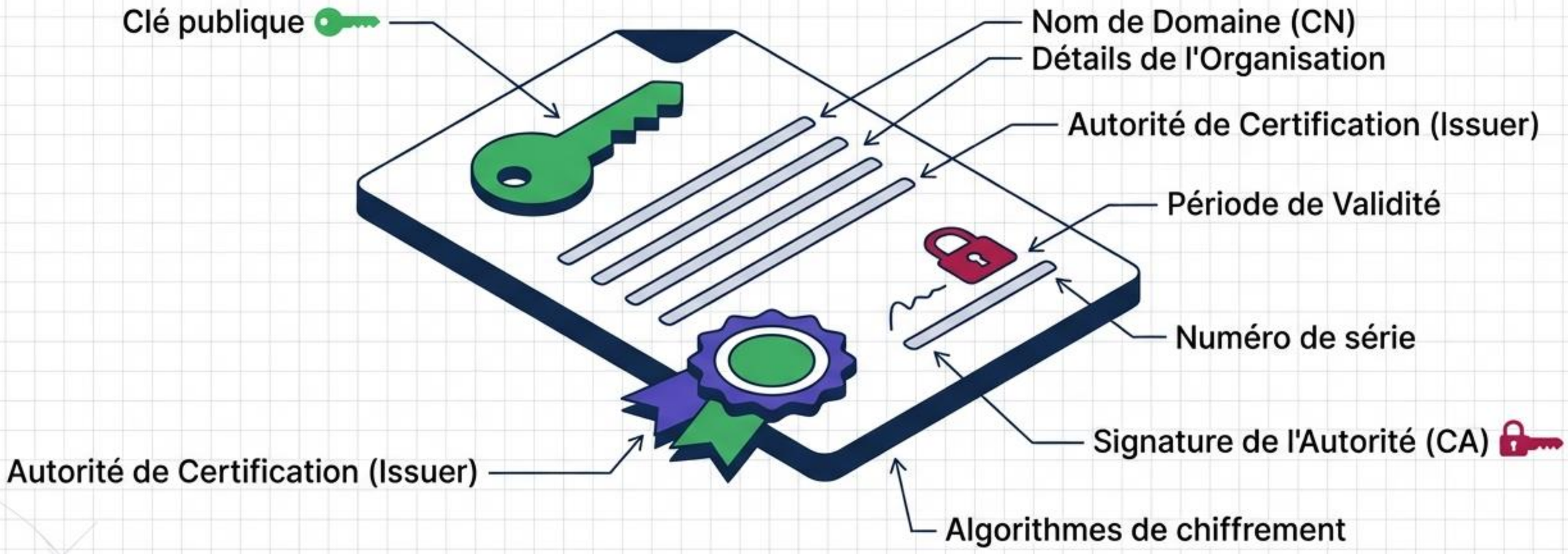
Un CA (Certificate Authority) est une entité de confiance qui délivre des certificats numériques permetant d'authentifier l'identité d'un serveur, d'un utilisateur ou d'un appareil. Il joue un rôle clé dans les infrastructures à clé publique (PKI).

Let's Encrypt DigiCert GlobalSign Entrust

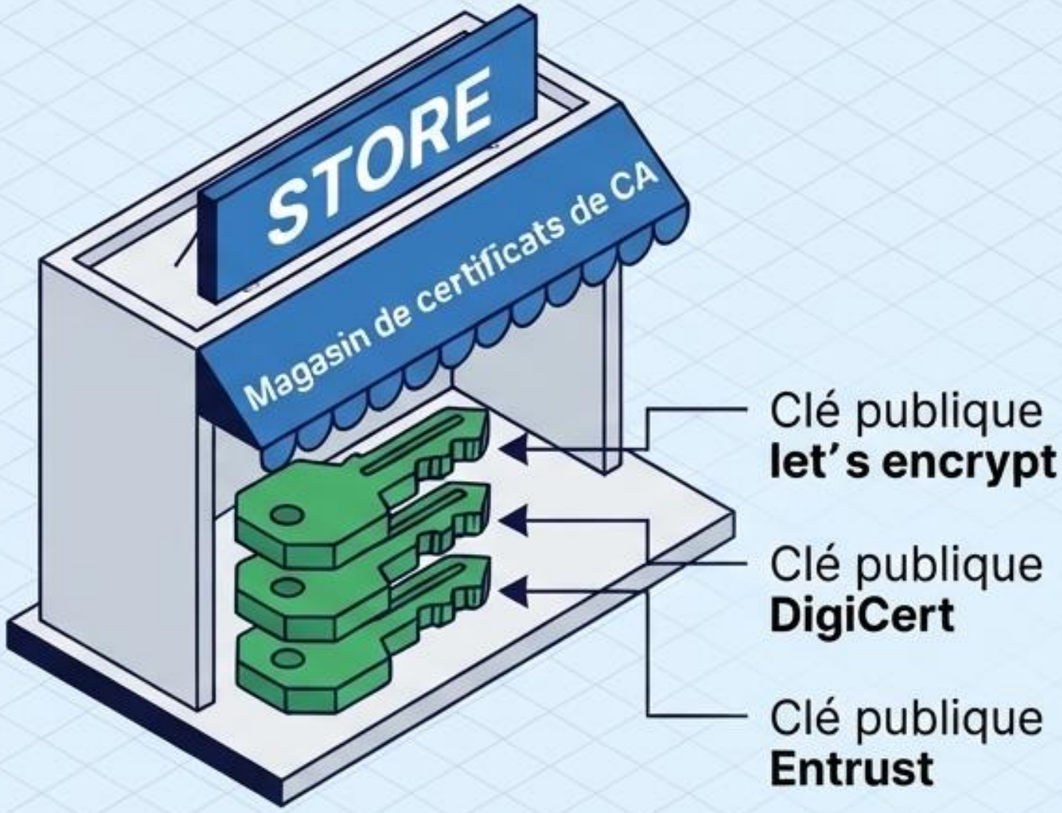
# La Signature d'un Certificat



# Anatomie d’un Certificat TLS



# La Vérification et le "Store"




Sur **Windows** :
Stocké dans le **Magasin de certificats Windows**.
Commande : **certmgr.msc**

Sur Linux :
**/etc/ssl/certs/** ou
**/usr/local/share/ca-certificates/**

# Mise en Pratique : Workflow OpenSSL

# Génération : CA et Clés

```
# 1. Création un certificat CA
openssl req -new -x509 -days 1826 -extensions
v3_ca -keyout ca.key -out ca.crt

# 2. Création d'une clé privée pour le serveur
openssl genrsa -out server.key 2048
```

# Génération : CSR et Signature

```
# 3. Création d'une demande de signature (CSR)
openssl req -out server.csr -key server.key -new

# 4. Signature et génération du certificat
openssl x509 -req -in server.csr -CA ca.crt -CAkey
ca.key -CAcreateserial -out server.crt -days 360
```

# Le Fichier server.crt : Encodage vs Réalité

## Format PEM (Base64)

```
-----BEGIN CERTIFICATE-----
MIIEvQIBADANBgkqhkiG9wBBAQEFAASCBKcwgggSjAgEAAoIBAQDfN19iPGjK
LcmM8U2mbwUIDsBaa648BUGdFAAIRNHUuNRNVARDPNVTRScaOHVd9SODIRGdC
AOkdUMzKoB7yIMBUGWBHBwRBAASOHHIZPoUN6AAwKHOBAAAoNNMnI9OOAM7K1
MIQZohURIB1MVAneBMPBgHxNU3AcvbvMI6FMBdVg1OAAAAEAAQUIn90SqH1BB
gAkgAEAAoOBwOBANBgkqNBAMIIVodqBUP3s8ANAMOBAAACAAMAwNIIRfOS8Cp
naBHaDANDBNOeABgkMkIQDIK39wjsrLBmC4RmzADgYdxSg3P11EonbH3S8e9BM
a3NeV0x6BbAgIQgAAANHwHEdYuTN1sQBAAAgAAA9wwIouKA03TEABDYUPToDUI
HUvgCUusGgSGrgRXkiTd4OdaABAOEkgmAA3dAgAAuMIQ0GG2StlwFRdf3zQA7o
ddCDv8AaA2oGHAAgo3daTSHtWlwKIcvZHwaRBaAAAY81aAADAGGv6226CmH7S1
2dWeauxTAO6UVUILUSwWVJ2uGAYbdSTMABOwKIB6ZBwmBd26CBOkTSNeN88Br
2gWeHAneVsGa389edSNA1ORU3TASv8FcRKFTWEvSUdaV2doA0OPoXAUBHQ9AA
ATwWn9XKZKLeZ9AU3WSPiaNNTAonoMz1dHiT9fotivozaABUsD1nHewKbnsa81
ackdV7w7NONBA4AvVAYwIMUxagIaUTAAMJoOHSHwBowH396AjMiFBM7pa007
nBARAoHMidSAAQGdB3AdIBHCanGDAxmBBFAAAs1wumbBYJTDAAABIBwOkGoBJ
wwnn0GK
-----END CERTIFICATE-----
```

## Décodé (Text)

```
# 1. Décoder le certificat
openssl x509 -in server.crt -text -noout
Certificate:
    Data:
      Version: 1 (0x0)
      Serial Number:
02:26:5b:ca:06:48:91:93:df:87:84:b1:01:87:5e:05:57:12:8c:90
      Signature Algorithm: sha256WithRSAEncryption
      Issuer: C = FR, ST = Some-State, O = Internet Widgits
Pty Ltd
      Validity
        Not Before: Feb 20 16:04:28 2025 GMT
        Not After : Feb 15 16:04:28 2026 GMT
    Subject:  C = FR, ST = Some-State, L = PARIS, O = Newton,
OU = CIEL, CN = 192.168.1.194, emailAddress = ciel@newton.fr
    Subject Public Key Info:
        Public Key Algorithm: rsaEncryption
        Public-Key: (2048 bit)
        Modulus: 00:df:36:9f:62:5c:68:ca:fa:ed:63:29:5d:c0:e5:
        Exponent: 65537 (0x10001)
```

# Mathématiques RSA : La Clé Publique

## n (Modulus) - Produit de deux nombres premiers

Modulus (2048 bit):
00:df:36:5f:62:3c:68:ca:fa:ed:63:29:5d:c0:e5:7f:3b:ab:72:d6:bf:bf:e3:78:14:70:5e:80:9c:dd:
26:13:19:37:6c:f3:e6:d7:8e:b1:6d:c6:1e:5a:dd:0f:8e:21:cd:21:91:fa:98:23:93:b5:52:28:6c:3b:
df:d2:3c:e4:f2:a9:b2:cd:e8:77:43:4c:0c:ff:2a:9b:2b:6d:97:c1:e5:3b:b8:3f:26:f5:35:79:b4:c1:0
b:52:72:a8:94:0f:cf:e9:7f:b5:5a:52:5d:41:51:46:e2:22:ba:4f:db:fd:2a:0a:5d:4b:3b:a8:3b:0a:a
a5:54:59:c3:66:33:ce:90a2:50:08:4e:d8:ff:cc:cc:7d:e9:73:1f:e9:91:8d:d4:ef:28:e7:31:5f:4a:b2:
e:43:59:70:e7:71:0a:12:57:36:da:b9:69:b3:19:57:c3:fb:7f:7c:2d:4e:70:94:c4:62:ca:b8:61:42:
28:97:15e:52:a2:10:9e:c7:ec:a0:8d:d7:87:77:66:aa:7c:af:f5:f3:60:7b:f2:57:bb:0d:88:5a:eb:7
7:56:28:ea:a4:76:27:b9:ca:1e:ed:65:8f:df:6e:3f:07:73:1f

## e (Exposant Public)

Exponent: 65537 (0x10001)

Nombre premier de Fermat (2^16 + 1).
En binaire : 1000000000000001.
Rapide à calculer.

# Mathématiques RSA : La Clé Privée

```
openssl rsa -in server.key -text -noout
```

**n (Modulus: (2048 bit)**

```
00:df:36:5f:62:3c:68:ca:fa:ed:63:29:5d:
c0:e5:7f:3b:ab:72:d6:bf:bf:e3:78:14:70:5
e:80:9c:dd:26:13:19:37:6c:f3:e6:d7:8e:b
1:6d:c6:1e:5a:dd:0f:8e:21:cd:21:91:fa:98:
23:93:b5:52:28:6c:3b:df:d2:3c:e4:f2:a9:
b2:cd:e8:77:43:4c:0c:ff:2a:9b:2b:6d:97:
c1:e5:3b:b8:3f:26:f5:35:79:b4:c1:0b:52:
72:a8:94:0f:cf:e9:7f:b5:5a:52:5d:41:51:
46:e2:22:ba:4f:db:fd:2a:0a:5d:4b:3b:a8:
3b:0a:a5:54:59:c3:66:33:ce:90:a2:54:0e
cb8:02:4e:d8:ff:cc:cc:7d:8e:89:cf:ca:f9:
e2:c9:S3:77:95:4b:88:49:ca:f7:e52:7e:
43:59:70:e7:71:0a:12:57:36:da:b9:69:b3:
19:57:c3:fb:7f:7c:2d:4e:70:94:c4:62:ca:
8:61:42:97:5e:87:ef:ef:2b:71:83:83:06:ab
:fc:de:fe:bb:d9:52:a2:10:9e:c7:ec:a0:8d:
d7:87:77:66:aa:7c:af:f5:f3:60:7b:f2:57:b
bb:0d:88:5a:eb:77:56:28:ea:a4:76:27:b9:
ca:1e:ed:65:8f:df:6e:3f:07:73:1f
```

**n (Modulus)**

Chiffrement = Message$^{e}$ mod n

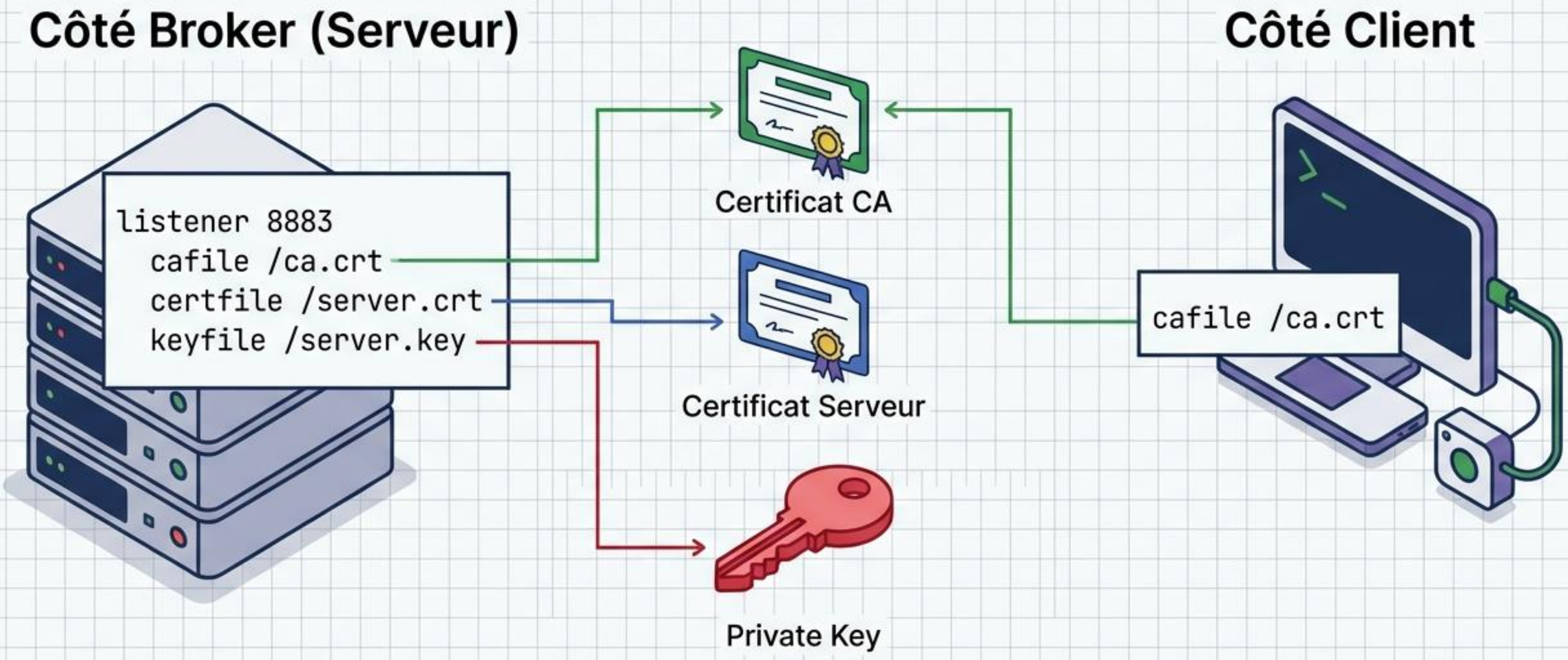
Déchiffrement = Chiffre$^{d}$ mod n

**privateExponent:**

```
11:ff:2c:49:9b:5d:47:a0:5b:ab:77:16:5c:0
b:24:eb:06:b3:c7:a4:c6:2f:ec:ea:98:f4:c
5:a1:e0:22:c5:bf:6e:bc:96:1d:b2:9b:8a:aa
:b3:22:97:68:e3:6c:2e:46:ca:02:1f:16:4d:
28:07:a2:8c:27:cd:dc:09:f3:66:f2:39:c4:
74:dd:9d:60:0b:ad:b0:95:1a:d5:b1:8c:5e:
d1c0:b2:e6:44:99:d8:a9:eb:6c:f5:20:bc:a
5:e2:ea:33:75:3c:1a:bf:21:8c:eb:4b:46:4f:
cd:45:00:53:58:62:1f:5b:ce:49:0d:13:55:
94:66:ec:a5:d8:83:ac:cf:14:60:0c:d5:2b:
b:2c:bf:8e:e0:d4:aa:98:52:8f:0d:e8:4b:91:
dd:9a:85:4d:6a:7c:f8:d5:10:80:7e:3c:a5
s1:14:d1:11:78:011:0b:7b:1e:3c:a3:f44:fc:
fc:4d:65:0c:ed:e6:a8:8e:af:1f:cf:8f:ca:27
7:07:92:fd:1f:fb:fd:d8:78:9d:5b:95:c8:dd
:64:ac:ef:32:4e:08:9b:d8:8a:04:6d:08:27:
32:c4:0d:4c:89:68:a5:87:80:33:0b:eb:55:
bb:92:f4:2e:25:8d:12:ff:41:c9:e6:bc:20:c
9:f1:5b:06:98:6f:62:dc:b6:a1
```

**d (Exposant Privé)**

# Application Réelle : MQTTS



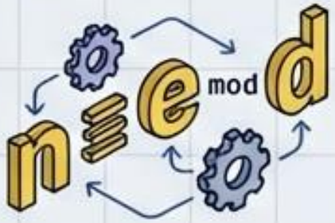
# En Résumé

**Fondation** : ISO 27001 & la Triade CIA.

**Outils** : Chiffrement asymétrique (RSA) et Hachage (SHA256).

**Confiance** : Le CA signe les certificats pour garantir l'identité.

**Pratique** : OpenSSL permet de générer la chaîne complète.




**Mathématiques** : La sécurité repose sur la relation entre n, e et d.